Guía de Actividades **Práctico-Experimentales** Nro. 003

1. Datos Generales

Asignatura	Sistemas Distribuidos
Ciclo	6to
Unidad	Uno
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	3
Título de la Práctica	Sincronización de Procesos: Exclusión Mutua, Semáforos y Problemas Clásicos de Concurrencia
Nombre del Docente	José O. Guamán Q.
Nombre de integrantes:	Marco Castillo Viviana Córdova Ariel Tandazo Isauro Rivera
Fecha	06 de octubre de de 2026 09 de octubre de 2026
Horario	Martes 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

Implementar una barrera de sincronización con variables de condición que detenga a los hilos hasta que todos alcancen el punto de encuentro.

- Implementar mecanismos de exclusión mutua utilizando mutex y cerrojos (locks) para proteger secciones críticas en programas multihilo, garantizando que no se produzcan condiciones de carrera al acceder a recursos compartidos.
- Aplicar semáforos como herramienta de sincronización para controlar el acceso a recursos limitados y coordinar la ejecución de hilos según restricciones específicas de orden y disponibilidad.
- Resolver el problema clásico del Productor-Consumidor utilizando buffers compartidos y mecanismos de sincronización que aseguren que el productor no escriba en un buffer lleno y el consumidor no lea de un buffer vacío.
- Implementar la solución al problema de Lectores-Escritores, distinguiendo entre las variantes con prioridad para lectores y

prioridad para escritores, analizando las implicaciones de cada enfoque en términos de inanición y concurrencia.

3. Materiales y reactivos:

Guia practica Lista de ejercicios

4. Equipos y herramientas

Hardware

- Computador

Software

- Lenguaje de programación (Python, java sprint boot, node)
- IDE de programación
- Archivo de texto

5. Procedimiento / Metodología

Link Github ejercicio 1 resuelto: https://github.com/YooneR1209/APE_03_Sistemas_Distribuidos

Ejercicio 1. Control de acceso a un estacionamiento — Semáforo contador

Objetivo: Aplicar semáforos contadores para controlar el acceso concurrente a un recurso limitado.

Situación:

Un estacionamiento dispone de **4 espacios**. Llegan 10 vehículos de manera concurrente. Cada vehículo debe esperar si no existe un espacio disponible. Cuando un vehículo sale, libera el espacio para otro vehículo.

Procedimiento

1. Identifique el recurso compartido.

   El recurso compartido son los espacios del estacionamiento. Son un recurso limitado (solo 4) al que compiten 10 vehículos a la vez.

```
    private final long maxMs;

    private final Semaphore espacios;               // semáforo contador
    private final ExecutorService pool;
    private final CountDownLatch terminado = new CountDownLatch(1);
```

2. Determine la cantidad máxima de vehículos que pueden permanecer dentro del estacionamiento.

   La cantidad máxima es 4 vehículos. Es la capacidad física del estacionamiento y es el valor con el que se inicializa el semáforo.

```
public class ParkingSimulation {

    /** Espacios reales del estacionamiento según la guía (criterio de verificación). */
    public static final int ESPACIOS_REALES = 4;

    public enum Estado { ESPERANDO, DENTRO, FUERA }
```

3. Cree un semáforo contador inicializado en 4.

Se creó un Semaphore con 4 permisos. Cada permiso representa un espacio libre. Es "contador" porque su valor va de 0 a 4, a diferencia de un semáforo binario o un mutex.

```
        this.minMs = minMs;
        this.maxMs = maxMs;
        this.espacios = new Semaphore(capacidad, true);   // true = FIFO (justo)
        this.pool = Executors.newFixedThreadPool(total);
        this.estados = new Estado[total + 1];
    }
```

4. Cree un proceso vehiculo(id).

```
    }

    private void vehiculo(int id) {
        try {
            registrar("Vehículo " + id + " llega al estacionamiento");
```

5. El vehículo deberá solicitar un espacio antes de ingresar.

```
    registrar("Vehículo " + id + " llega al estacionamiento");

    espacios.acquire();    // wait(espacios): se bloquea si no hay espacio
    try {
        synchronized (lock) {
```

6. Si no existen espacios disponibles, deberá permanecer bloqueado.
7. Una vez que obtenga un espacio, deberá simular su permanencia.
8.

```
    }

    Thread.sleep(ThreadLocalRandom.current().nextLong(minMs, maxMs + 1));   // permanencia

    synchronized (lock) {
```

9. Al salir, deberá liberar el espacio mediante signal().

```
        }
    } finally {
        espacios.release();                    // signal(espacios)
    }
atch (InterruptedException e) {
```

10. Cree 10 procesos vehículo.



11. Analice qué sucede cuando llegan más de 4 vehículos simultáneamente.

Análisis: Cuando llegan más de 4 vehículos simultáneamente entran los 4 primeros mientras que los siguiente esperan que se vaya liberando espacios en el estacionamiento para ocupar el lugar, en FIFO.

Pseudocódigo

```
SEMÁFORO espacios = 4
PROCEDIMIENTO vehiculo(id)

    escribir("Vehículo ", id, " llega al estacionamiento")

    wait(espacios)

    escribir("Vehículo ", id, " ingresa al estacionamiento") estacionar(id)

    esperar_tiempo()

    escribir("Vehículo ", id, " sale del estacionamiento")

    signal(espacios)

FIN PROCEDIMIENTO


INICIO

    PARA i = 1 HASTA 10 HACER
        crear_proceso(vehiculo, i) FIN
    PARA

FIN
```

Pregunta de análisis:

¿Qué ocurriría si el semáforo se inicializara en 1? ¿Y si se inicializara en 10?

- Si el semáforo se inicializa en 1 la cola se vuelve lenta ya que únicamente se podría atender a 1 vehículo a la vez.
- Si se inicializa en 10 el estacionamiento podría albergar un mayor número de autos sin que la cola se vuelva demasiado larga.

Ejercicio 2. Cajero automático compartido — Exclusión mutua

Objetivo: Implementar exclusión mutua para evitar condiciones de carrera sobre un recurso compartido.

Situación:

Una cuenta bancaria tiene un saldo inicial de $1000. Tres procesos realizan operaciones simultáneamente sobre la misma cuenta:

- Proceso 1: retiro de $300.
- Proceso 2: retiro de $250.
- Proceso 3: depósito de $200.

El saldo debe modificarse de forma segura. Procedimiento

1. Identifique la variable compartida.

```
private volatile int saldo = SALDO_INICIAL;
```

```java
public Operacion retirar(String proceso, int monto) {
    validarMonto(monto);
    lock.lock();
    try {
        registrar(proceso, "Intenta retirar $" + monto, saldo);
        if (saldo >= monto) {
            saldo = saldo - monto;
            registrar(proceso, "Retiro realizado de $" + monto, saldo);
            return new Operacion(proceso, tipo: "RETIRO", monto, realizada: true, saldo);
        }
        registrar(proceso, "Saldo insuficiente para retirar $" + monto, saldo);
        return new Operacion(proceso, tipo: "RETIRO", monto, realizada: false, saldo);
    } finally {
        lock.unlock();
    }
}
```

2. Determine qué operaciones pueden ejecutarse simultáneamente.

```java
List<Callable<Operacion>> procesos = List.of(
        proceso(salida, () -> conMutex
                ? cuenta.retirar(proceso: "Proceso 1", RETIRO_PROCESO_1)
                : cuenta.retirarSinMutex(proceso: "Proceso 1", RETIRO_PROCESO_1)),
        proceso(salida, () -> conMutex
                ? cuenta.retirar(proceso: "Proceso 2", RETIRO_PROCESO_2)
                : cuenta.retirarSinMutex(proceso: "Proceso 2", RETIRO_PROCESO_2)),
        proceso(salida, () -> conMutex
                ? cuenta.depositar(proceso: "Proceso 3", DEPOSITO_PROCESO_3)
                : cuenta.depositarSinMutex(proceso: "Proceso 3", DEPOSITO_PROCESO_3)));
```

3. Cree un mutex para proteger el saldo.

```java
public Operacion retirar(String proceso, int monto) {
    validarMonto(monto);
    lock.lock();
    try {
        registrar(proceso, "Intenta retirar $" + monto, saldo);
        if (saldo >= monto) {
            saldo = saldo - monto;
            registrar(proceso, "Retiro realizado de $" + monto, saldo);
            return new Operacion(proceso, tipo: "RETIRO", monto, realizada: true, saldo);
        }
        registrar(proceso, "Saldo insuficiente para retirar $" + monto, saldo);
        return new Operacion(proceso, tipo: "RETIRO", monto, realizada: false, saldo);
    } finally {
        lock.unlock();
    }
}
```

4. Antes de consultar o modificar el saldo, el proceso debe adquirir el mutex.

```java
@PostMapping("/simular/con-mutex")
public ResultadoSimulacion simularConMutex() {
    return cajeroService.ejecutarSimulacion(conMutex: true);
}

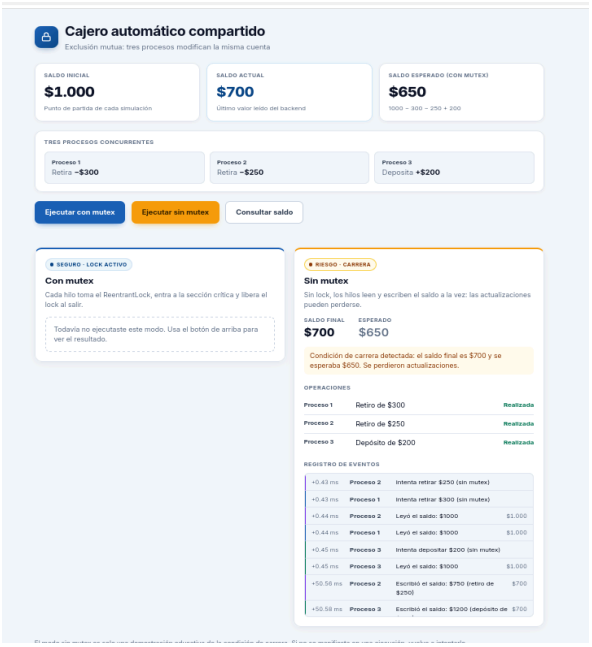
@PostMapping("/simular/sin-mutex")
public ResultadoSimulacion simularSinMutex() {
    return cajeroService.ejecutarSimulacion(conMutex: false);
}
```

5. Realice la operación correspondiente.
6. Libere el mutex después de actualizar el saldo.

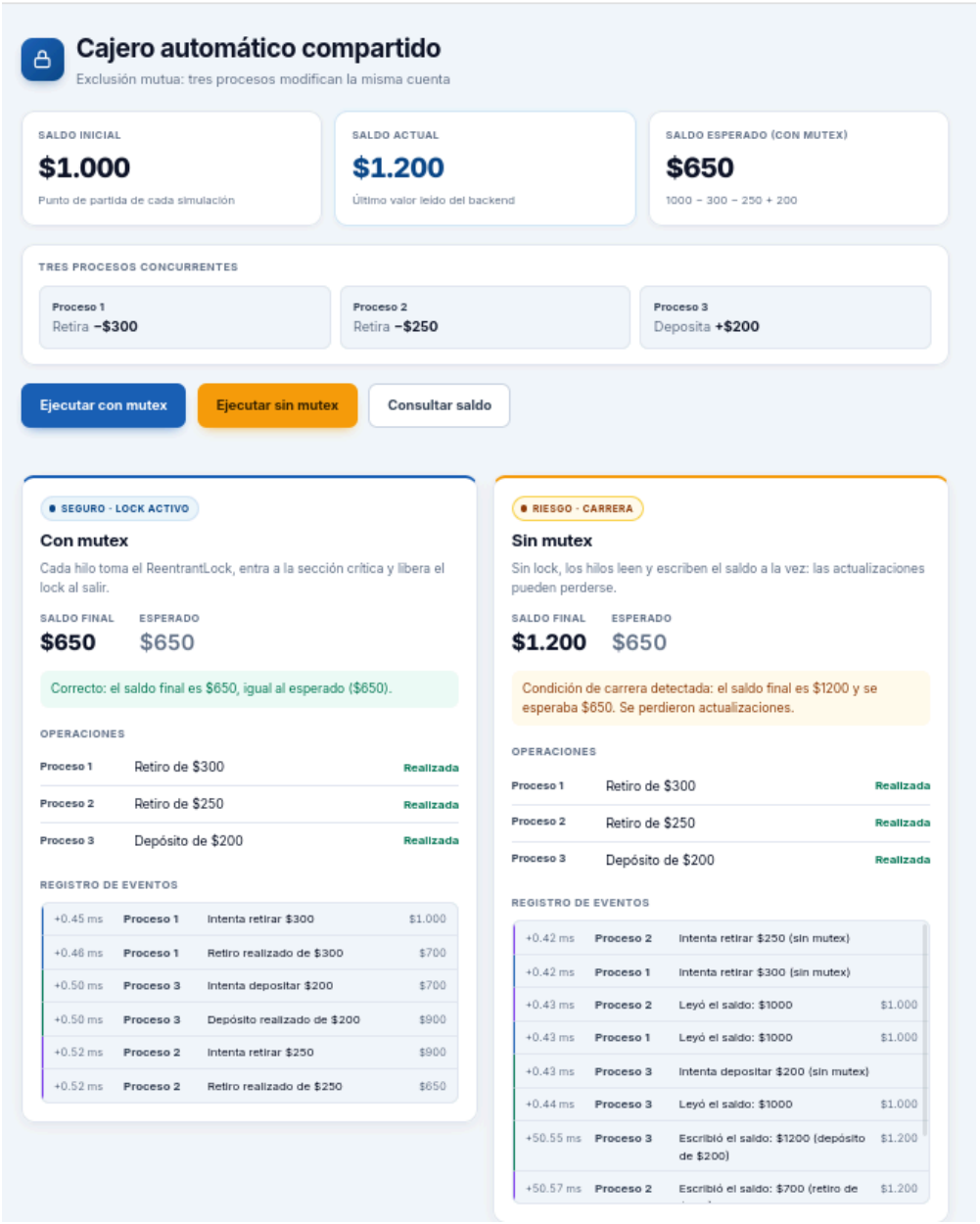
```java
public Operacion depositar(String proceso, int monto) {
    validarMonto(monto);
    lock.lock();
    try {
        registrar(proceso, "Intenta depositar $" + monto, saldo);
        saldo = saldo + monto;
        registrar(proceso, "Depósito realizado de $" + monto, saldo);
        return new Operacion(proceso, tipo: "DEPOSITO", monto, realizada: true, saldo);
    } finally {
        lock.unlock();
    }
}
```

7. Cree los tres procesos.

8. Ejecute el programa varias veces.
9. Analice qué podría ocurrir si se elimina el mutex.



10. Compare el comportamiento con y sin exclusión mutua.



Pseudocódigo

```
saldo = 1000

MUTEX mutex_saldo = 1


PROCEDIMIENTO retirar(id, cantidad) lock(mutex_saldo)

    escribir("Proceso ", id, " intenta retirar $", cantidad)

    SI saldo >= cantidad ENTONCES saldo =

        saldo - cantidad

        escribir("Retiro realizado") escribir("Saldo actual:
        $", saldo)
```

```
    SINO
        escribir("Saldo insuficiente") FIN SI
    unlock(mutex_saldo)       FIN
PROCEDIMIENTO

PROCEDIMIENTO depositar(id, cantidad) lock(mutex_saldo)
    escribir("Proceso ", id, " deposita $", cantidad) saldo =
    saldo + cantidad
```

escribir("Saldo actual: $", saldo) unlock(mutex_saldo)

FIN PROCEDIMIENTO

INICIO

crear_proceso(retirar, 1, 300)

crear_proceso(retirar, 2, 250)
crear_proceso(depositar, 3, 200) FIN

Pregunta de análisis:

¿Por qué el acceso a saldo debe protegerse con un mutex?

El acceso de saldo debe protegerse con mutex ya que los tres cajeros o procesos del sistema, pueden retirar o depositar dinero al mismo tiempo. Sin esta protección pueden llegar a ocurrir las condiciones de carrera, en las que una operación sobrescribe el resultado de otra y el saldo final resulta incorrecto. La implementación de mutuo, permite que un único hilo a la vez ejecute la consulta, retiro o depósito.

Ejercicio 3. Fábrica de ensamblaje — Productor-Consumidor

Objetivo: Implementar el patrón productor-consumidor utilizando una cola limitada y semáforos.

Situación:

Una fábrica tiene una línea de producción con capacidad para almacenar **5 piezas** temporalmente.

- Los productores fabrican piezas.
- Los consumidores toman piezas para ensamblarlas.
- Un productor no puede colocar una pieza si el almacén está lleno.
- Un consumidor no puede retirar una pieza si el almacén está vacío.

Procedimiento

1. Cree una cola con capacidad máxima de 5 elementos.
2. Cree un semáforo espacios inicializado en 5.
3. Cree un semáforo elementos inicializado en 0.
4. Cree un mutex para proteger la cola.
5. Implemente el proceso productor.
6. El productor deberá esperar un espacio disponible.
7. Debe adquirir el mutex antes de modificar la cola.
8. Debe insertar una pieza.
9. Debe liberar el mutex.
10. Debe indicar que existe una nueva pieza.
11. Implemente el proceso consumidor.
12. El consumidor deberá esperar hasta que exista una pieza.
13. Debe adquirir el mutex antes de retirar una pieza.

14. Debe retirar la pieza.
15. Debe liberar el mutex.
16. Debe indicar que existe un espacio disponible.

Pseudocódigo

```
COLA almacen
CAPACIDAD = 5

SEMÁFORO espacios = 5 SEMÁFORO
elementos = 0

MUTEX mutex_almacen = 1


PROCEDIMIENTO productor(id)

REPETIR 5 VECES

    pieza = fabricar_pieza()

    wait(espacios)

    lock(mutex_almacen)

    insertar(almacen, pieza)

    escribir("Productor ", id,
        " agregó pieza ", pieza)

    unlock(mutex_almacen)

    signal(elementos)

  FIN REPETIR

FIN PROCEDIMIENTO


    PROCEDIMIENTO consumidor(id)

REPETIR 5 VECES

    wait(elementos)
```

```
        lock(mutex_almacen)   pieza   =

        extraer(almacen)

        escribir("Consumidor   ",   id,   "
              retiró pieza ", pieza)

        unlock(mutex_almacen)

        signal(espacios) ensamblar(pieza)

    FIN REPETIR

FIN PROCEDIMIENTO


INICIO

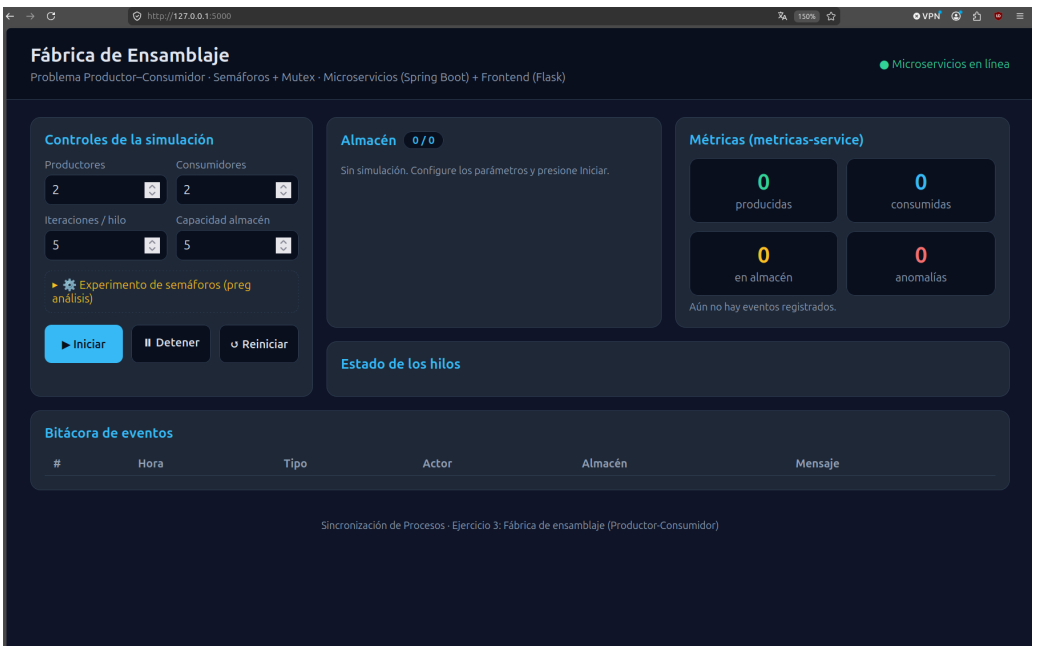
    PARA i = 1 HASTA 2 HACER
        crear_proceso(productor,    i)    FIN
    PARA

    PARA i = 1 HASTA 2 HACER
        crear_proceso(consumidor,    i)    FIN
    PARA

FIN
```

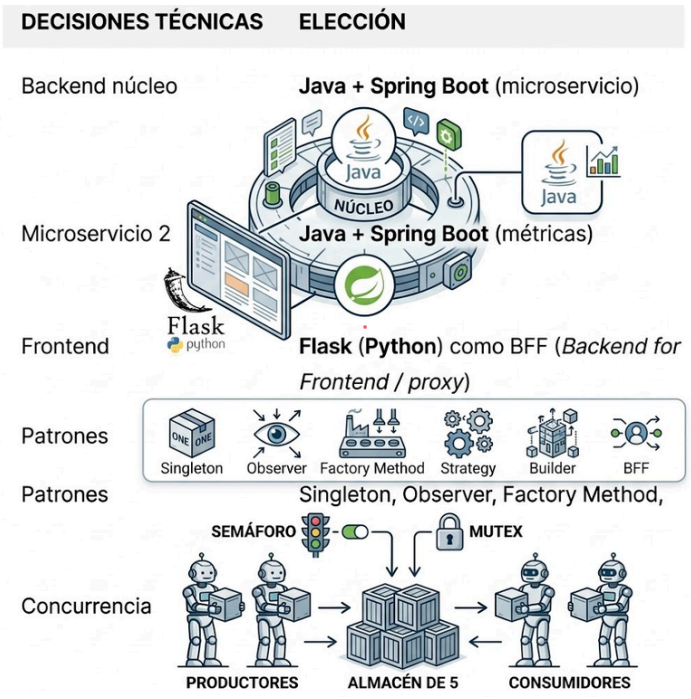
DESARROLLO:

Dashboard:



Estructura del proyecto:

```
fabrica-ensamblaje/
├── simulacion-service/ # Microservicio 1 – Spring Boot (puerto 8080) → NÚCLEO
│   ├── pom.xml
│   └── src/main/
│       └── java/com/fabrica/simulacion/
│           ├── SimulacionApplication.java
│           ├── config/WebConfig.java
│           ├── controller/SimulacionController.java
│           ├── controller/EventosController.java
│           ├── service/FabricaSimulacion.java
│           ├── core/ ← CONCURRENCIA (semáforos, mutex, hilos)
│           ├── observer/ ← Patrón Observer
│           ├── factory/ ← Patrón Factory Method
│           ├── model/
│           ├── dto/
│           └── resources/application.yml
├── metricas-service/ # Microservicio 2 – Spring Boot (puerto 8081)
│   └── pom.xml  src/main/java/com/abrica/metricas/...
└── frontend-flask/ # Frontend – Flask (puerto 5000)
    ├── app.py  requirements.txt  templates/index.html  static/  ├── css/styles.css
    └── requirements.txt  templates/index.html  └── js/app.js
```

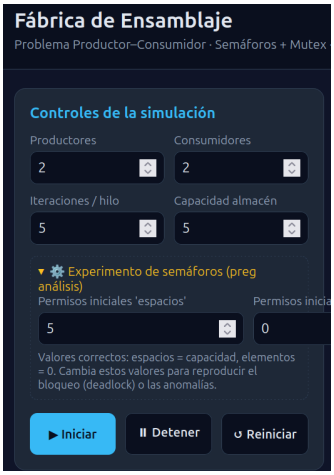


6. PRUEBAS Y RESULTADOS

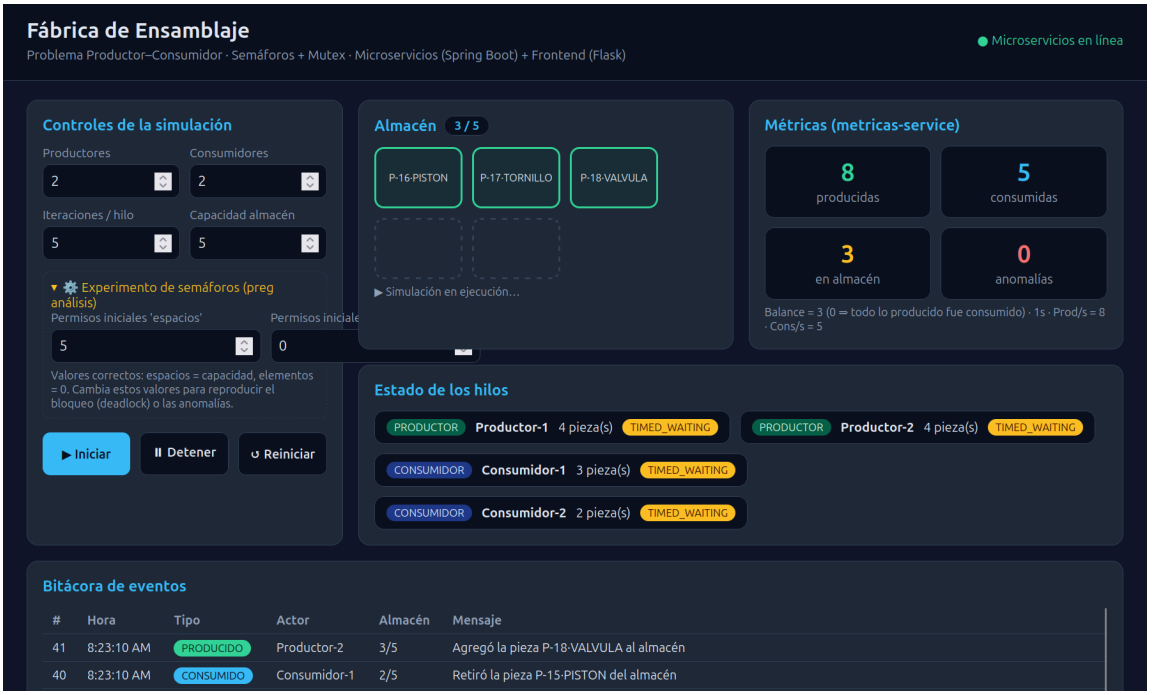
6.1 Prueba 1. Escenario base:

Configuración: Productores=2, Consumidores=2, Iteraciones=5, Capacidad=5, *espacios=5, elementos=0.*

Dashboard configurado con las métricas solicitadas:



Ejecución:



Métricas finales tras la ejecución (balance en 0):



Resultados obtenidos:

Métrica	Esperado	Obtenido
Piezas producidas	10	[10]
Piezas consumidas	10	[10]
Balance (producidas − consumidas)	0	[0]
Anomalías	0	[0]
Nivel máximo observado del almacén	≤ 5 (nunca 6)	[5]
Nivel final del almacén	0	[0]

Bitácora de eventos PRODUCIDO/CONSUMIDO con nivel ≤ 5

INICIO de simulación:

Bitácora de eventos

#	Hora	Tipo	Actor	Almacén	Mensaje
38	8:23:10 AM	PRODUCIDO	Productor-1	2/5	Agregó la pieza P-16·PISTON al almacén
37	8:23:09 AM	CONSUMIDO	Consumidor-2	1/5	Retiró la pieza P-14·ENGRAJE del almacén
36	8:23:09 AM	PRODUCIDO	Productor-2	2/5	Agregó la pieza P-15·PISTON al almacén
35	8:23:09 AM	PRODUCIDO	Productor-1	1/5	Agregó la pieza P-14·ENGRAJE al almacén
34	8:23:09 AM	CONSUMIDO	Consumidor-1	0/5	Retiró la pieza P-13·CARCASA del almacén
33	8:23:09 AM	PRODUCIDO	Productor-1	1/5	Agregó la pieza P-13·CARCASA al almacén
32	8:23:09 AM	CONSUMIDO	Consumidor-2	0/5	Retiró la pieza P-12·TORNILLO del almacén
31	8:23:09 AM	PRODUCIDO	Productor-2	1/5	Agregó la pieza P-12·TORNILLO al almacén
30	8:23:08 AM	CONSUMIDO	Consumidor-1	0/5	Retiró la pieza P-11·CARCASA del almacén
29	8:23:08 AM	PRODUCIDO	Productor-1	1/5	Agregó la pieza P-11·CARCASA al almacén
28	8:23:08 AM	INICIO	SISTEMA	—	Simulación iniciada → Productores=2, Consumidores=2, Iteraciones=5, Capacidad=5, espacios=5, elementos=0

FIN de la simulación:

Bitácora de eventos

#	Hora	Tipo	Actor	Almacén	Mensaje
53	8:23:12 AM	FIN	SISTEMA	—	Simulación finalizada → producidas=10, consumidas=10, anomalías=0
52	8:23:12 AM	INFO	Consumidor-1	—	Terminó: ensambló 5 pieza(s)
51	8:23:11 AM	INFO	Consumidor-2	—	Terminó: ensambló 5 pieza(s)
50	8:23:11 AM	CONSUMIDO	Consumidor-1	0/5	Retiró la pieza P-20·CARCASA del almacén
49	8:23:11 AM	CONSUMIDO	Consumidor-2	1/5	Retiró la pieza P-19·PISTON del almacén
48	8:23:10 AM	CONSUMIDO	Consumidor-2	2/5	Retiró la pieza P-18·VALVULA del almacén
47	8:23:10 AM	CONSUMIDO	Consumidor-1	3/5	Retiró la pieza P-17·TORNILLO del almacén
46	8:23:10 AM	INFO	Productor-2	—	Terminó: fabricó 5 pieza(s)
45	8:23:10 AM	PRODUCIDO	Productor-2	4/5	Agregó la pieza P-20·CARCASA al almacén
44	8:23:10 AM	CONSUMIDO	Consumidor-2	3/5	Retiró la pieza P-16·PISTON del almacén

Análisis: todos los productos producidos son consumidos, el almacén nunca supera 5 ni queda inconsistente, además en la bitácora cada evento PRODUCIDO muestra nivel ≤ 5, no se registró ninguna anomalía y el balance final fue 0.

Pregunta de análisis:

¿Qué sucede si espacios se inicializa en 0? ¿Qué sucede si elementos se inicializa en 5?

"Espacios" Inicializado en 0:



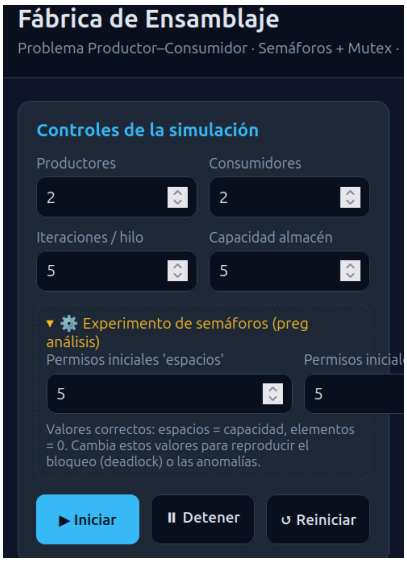


Resultado: los cuatro hilos quedan bloqueados de forma permanente debido a un interbloqueo total del sistema.

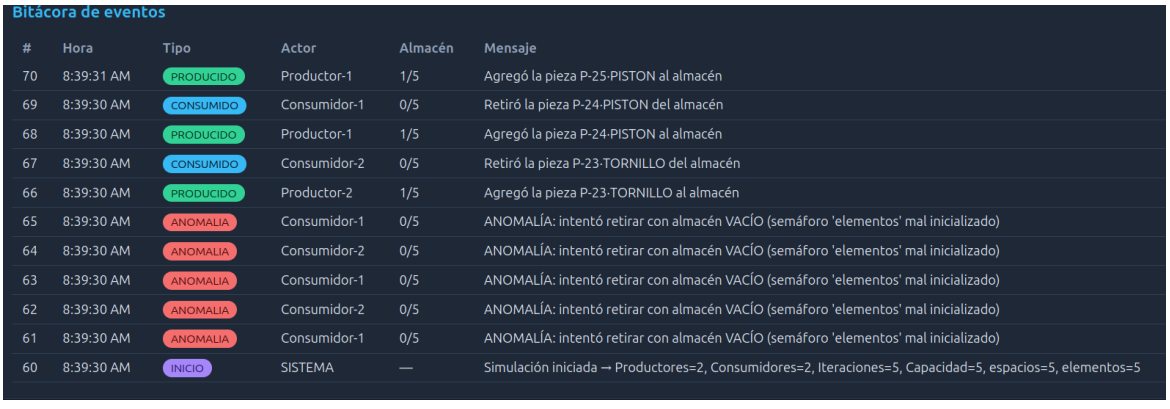
Técnicamente no es un deadlock por monitores que la JVM reporte, sino un bloqueo por semáforos sin fuente de permisos, funcionalmente equivalente: el sistema queda paralizado y solo una interrupción externa lo recupera.

Conclusión: sin espacios declarados, nadie puede producir; sin producción, nadie puede consumir.

“Elementos (permisos iniciales)” Inicializado en 5:







Bitácora de eventos

#	Hora	Tipo	Actor	Almacén	Mensaje
70	8:39:31 AM	PRODUCIDO	Productor-1	1/5	Agregó la pieza P-25·PISTON al almacén
69	8:39:30 AM	CONSUMIDO	Consumidor-1	0/5	Retiró la pieza P-24·PISTON del almacén
68	8:39:30 AM	PRODUCIDO	Productor-1	1/5	Agregó la pieza P-24·PISTON al almacén
67	8:39:30 AM	CONSUMIDO	Consumidor-2	0/5	Retiró la pieza P-23·TORNILLO del almacén
66	8:39:30 AM	PRODUCIDO	Productor-2	1/5	Agregó la pieza P-23·TORNILLO al almacén
65	8:39:30 AM	ANOMALIA	Consumidor-1	0/5	ANOMALÍA: intentó retirar con almacén VACÍO (semáforo 'elementos' mal inicializado)
64	8:39:30 AM	ANOMALIA	Consumidor-2	0/5	ANOMALÍA: intentó retirar con almacén VACÍO (semáforo 'elementos' mal inicializado)
63	8:39:30 AM	ANOMALIA	Consumidor-1	0/5	ANOMALÍA: intentó retirar con almacén VACÍO (semáforo 'elementos' mal inicializado)
62	8:39:30 AM	ANOMALIA	Consumidor-2	0/5	ANOMALÍA: intentó retirar con almacén VACÍO (semáforo 'elementos' mal inicializado)
61	8:39:30 AM	ANOMALIA	Consumidor-1	0/5	ANOMALÍA: intentó retirar con almacén VACÍO (semáforo 'elementos' mal inicializado)
60	8:39:30 AM	INICIO	SISTEMA	—	Simulación iniciada → Productores=2, Consumidores=2, Iteraciones=5, Capacidad=5, espacios=5, elementos=5



Bitácora de eventos

#	Hora	Tipo	Actor	Almacén	Mensaje
85	8:39:33 AM	FIN	SISTEMA	—	Simulación finalizada → producidas=10, consumidas=5, anomalías=5
84	8:39:32 AM	INFO	Productor-2	—	Terminó: fabricó 5 pieza(s)
83	8:39:32 AM	PRODUCIDO	Productor-2	5/5	Agregó la pieza P-32·CARCASA al almacén
82	8:39:32 AM	PRODUCIDO	Productor-2	4/5	Agregó la pieza P-31·ENGRAJE al almacén
81	8:39:32 AM	INFO	Consumidor-2	—	Terminó: ensambló 3 pieza(s)
80	8:39:32 AM	INFO	Productor-1	—	Terminó: fabricó 5 pieza(s)
79	8:39:32 AM	PRODUCIDO	Productor-1	3/5	Agregó la pieza P-30·VALVULA al almacén

Resultado: En el sistema se presenta una situación donde hasta cinco consumidores logran superar la espera sin que haya piezas disponibles, accediendo al almacén vacío y generando una condición de carrera.

Aunque una implementación sin protección arrojaría errores o datos basura, aquí el acceso está resguardado por un mutex y validaciones defensivas que detectan e identifican exactamente cinco anomalías sin realizar la extracción.

Como consecuencia, estas adquisiciones consumen iteraciones reales del proceso, lo que deja al finalizar la simulación piezas sin consumir y un balance distinto de cero, alterando el invariante principal del patrón.

Ejercicio 4. Cruce de vehículos — Control de acceso en una dirección

Objetivo: Coordinar procesos que comparten un recurso y evitar que dos grupos incompatibles accedan simultáneamente.

Situación:

Un puente permite circular en ambos sentidos, pero solamente puede existir **una dirección de circulación activa a la vez**.

Pueden existir varios vehículos cruzando hacia el norte, pero no puede haber vehículos hacia el norte y hacia el sur simultáneamente.

Procedimiento

1. Identifique el recurso compartido
   - **Descripción:** El recurso compartido es el puente de carril único por el cual los vehículos de ambos sentidos (Norte y Sur) deben circular de forma exclusiva para evitar colisiones.
   - **Archivo:** backend/src/main/java/com/puente/backend/modelo/Puente.java
   - **Fragmento de código:**

```java
public class Puente {
    private String direccion = "NINGUNA";
    private int vehiculosNorte = 0;
    private int vehiculosSur = 0;
    private int conteoConsecutivoNorte = 0;
    private int conteoConsecutivoSur = 0;

    public Puente() {}
```

2. Defina una variable que indique la dirección actual.

   Descripción: Se define la variable direccion inicializada en "NINGUNA", la cual rastrea qué sentido tiene el control activo del puente ("NORTE", "SUR" o "NINGUNA").

   Archivo: backend/src/main/java/com/puente/backend/modelo/Puente.java

   Fragmento de código:

```
public String getDireccion() { return direccion; }
public void setDireccion(String direccion) { this.direccion = direccion; }

public int getVehiculosNorte() { return vehiculosNorte; }
public void setVehiculosNorte(int vehiculosNorte) { this.vehiculosNorte = vehiculosNorte; }
```

3. Mantenga un contador de vehículos que están cruzando.

   Descripción: Se mantienen contadores enteros (vehiculosNorte y vehiculosSur) para rastrear cuántos vehículos se encuentran físicamente transitando por el puente en un momento dado.

   Archivo: backend/src/main/java/com/puente/backend/modelo/Puente.java

   Fragmento de código:

```
private int vehiculosNorte = 0;
private int vehiculosSur = 0;
```

```
public int getVehiculosNorte() { return vehiculosNorte; }
public void setVehiculosNorte(int vehiculosNorte) { this.vehiculosNorte = vehiculosNorte; }

public int getVehiculosSur() { return vehiculosSur; }
public void setVehiculosSur(int vehiculosSur) { this.vehiculosSur = vehiculosSur; }
```

4. Proteja las variables mediante un mutex.

   Descripción: Se implementa un ReentrantLock denominado mutex para asegurar que las operaciones críticas de lectura y modificación del estado del puente sean atómicas y libres de condiciones de carrera.

   Archivo: backend/src/main/java/com/puente/backend/servicio/PuenteService.java

   Fragmento de código:

```
@Service
public class PuenteService {

    // MUTEX mutex = 1
    private final Lock mutex = new ReentrantLock();

    // Estado del recurso compartido (direccion = NINGUNA, vehiculos_norte = 0, vehiculos_sur = 0)
    private final Puente estado = new Puente();

    // RETO: Evitar que una dirección monopolice el puente (Máximo 6 vehículos seguidos)
    private static final int MAX_CONSECUTIVOS = 6;

    public Puente obtenerEstado() {
        mutex.lock();
        try {
            return estado;
        } finally {
            mutex.unlock();
        }
    }
```

5. Un vehículo deberá verificar la dirección antes de entrar.

   Descripción: Al invocar el procedimiento de cruce, el hilo adquiere el mutex y evalúa mediante una condición si el sentido actual del puente es incompatible con su dirección.

   Archivo: backend/src/main/java/com/puente/backend/servicio/PuenteService.java

   Fragmento de código:

```java
public String vehiculoNorte(String id) throws InterruptedException {
    // lock(mutex)
    mutex.lock();
    try {
        // MIENTRAS direccion == SUR (o se cumplió el límite de 6 y hay vehículos esperando en el SUR) HACER esperar()
        while (estado.getDireccion().equals("SUR") ||
              (estado.getConteoConsecutivoNorte() >= MAX_CONSECUTIVOS && estado.getVehiculosSur() > 0)) {
            mutex.unlock();
            Thread.sleep(200); // esperar()
            mutex.lock();
        }

        // direccion = NORTE
        estado.setDireccion("NORTE");
        // vehiculos_norte = vehiculos_norte + 1
        estado.setVehiculosNorte(estado.getVehiculosNorte() + 1);

        // Contador del reto anti-monopolización
        estado.setConteoConsecutivoNorte(estado.getConteoConsecutivoNorte() + 1);
        estado.setConteoConsecutivoSur(0);

    } finally {
        // unlock(mutex)
        mutex.unlock();
    }
```

```java
public String vehiculoSur(String id) throws InterruptedException {
    // lock(mutex)
    mutex.lock();
    try {
        // MIENTRAS direccion == NORTE (o se cumplió el límite de 6 y hay vehículos esperando en el NORTE) HACER esperar()
        while (estado.getDireccion().equals("NORTE") ||
              (estado.getConteoConsecutivoSur() >= MAX_CONSECUTIVOS && estado.getVehiculosNorte() > 0)) {
            mutex.unlock();
            Thread.sleep(200); // esperar()
            mutex.lock();
        }

        // direccion = SUR
        estado.setDireccion("SUR");
        // vehiculos_sur = vehiculos_sur + 1
        estado.setVehiculosSur(estado.getVehiculosSur() + 1);

        // Contador del reto anti-monopolización
        estado.setConteoConsecutivoSur(estado.getConteoConsecutivoSur() + 1);
        estado.setConteoConsecutivoNorte(0);

    } finally {
        // unlock(mutex)
        mutex.unlock();
    }
```

6. Si existen vehículos en dirección contraria, deberá esperar.

   Descripción: Si el puente está ocupado por el sentido opuesto, el hilo libera temporalmente el mutex, entra en espera activa controlada (Thread.sleep) y reintenta cuando se libere.

   Archivo: backend/src/main/java/com/puente/backend/servicio/PuenteService.java

   Fragmento de código:

```java
while (estado.getDireccion().equals("SUR") ||
      (estado.getConteoConsecutivoNorte() >= MAX_CONSECUTIVOS && estado.getVehiculosSur() > 0)) {
    mutex.unlock();
    Thread.sleep(200); // esperar()
    mutex.lock();
}
```

```java
while (estado.getDireccion().equals("NORTE") ||
      (estado.getConteoConsecutivoSur() >= MAX_CONSECUTIVOS && estado.getVehiculosNorte() > 0)) {
    mutex.unlock();
    Thread.sleep(200); // esperar()
    mutex.lock();
}
```

7. Si puede entrar, incrementará el contador correspondiente.

   Descripción: Una vez superada la fase de espera, el hilo fija la dirección del puente y

suma +1 al contador del grupo que ingresa.

Archivo: backend/src/main/java/com/puente/backend/servicio/PuenteService.java

Fragmento de código:

```
    // direccion = NORTE
    estado.setDireccion("NORTE");
    // vehiculos_norte = vehiculos_norte + 1
    estado.setVehiculosNorte(estado.getVehiculosNorte() + 1);

    // Contador del reto anti-monopolización
    estado.setConteoConsecutivoNorte(estado.getConteoConsecutivoNorte() + 1);
    estado.setConteoConsecutivoSur(0);

} finally {
    // unlock(mutex)
    mutex.unlock();
}
```

```
    // direccion = SUR
    estado.setDireccion("SUR");
    // vehiculos_sur = vehiculos_sur + 1
    estado.setVehiculosSur(estado.getVehiculosSur() + 1);

    // Contador del reto anti-monopolización
    estado.setConteoConsecutivoSur(estado.getConteoConsecutivoSur() + 1);
    estado.setConteoConsecutivoNorte(0);

} finally {
    // unlock(mutex)
    mutex.unlock();
}
```

8. Después de cruzar, disminuirá el contador.

   Descripción: Una vez finalizado el tiempo de tránsito por el puente, el hilo vuelve a bloquear el mutex para restar -1 al contador del sentido correspondiente.

   Archivo: backend/src/main/java/com/puente/backend/servicio/PuenteService.java

   Fragmento de código:

```
// lock(mutex)
mutex.lock();
try {
    // vehiculos_norte = vehiculos_norte - 1
    estado.setVehiculosNorte(estado.getVehiculosNorte() - 1);
```

```
// lock(mutex)
mutex.lock();
try {
    // vehiculos_sur = vehiculos_sur - 1
    estado.setVehiculosSur(estado.getVehiculosSur() - 1);
```

9. Cuando no queden vehículos, el puente podrá cambiar de dirección.

Descripción: Si tras la salida del vehículo el contador llega a cero, la dirección del puente se restablece a "NINGUNA", permitiendo que el sentido opuesto pueda tomar posesión del recurso.

Archivo: backend/src/main/java/com/puente/backend/servicio/PuenteService.java

Fragmento de código:

```
    // SI vehiculos_norte == 0 ENTONCES direccion = NINGUNA
    if (estado.getVehiculosNorte() == 0) {
        estado.setDireccion("NINGUNA");
    }
} finally {
    // unlock(mutex)
    mutex.unlock();
}
```

```
    // SI vehiculos_sur == 0 ENTONCES direccion = NINGUNA
    if (estado.getVehiculosSur() == 0) {
        estado.setDireccion("NINGUNA");
    }
} finally {
    // unlock(mutex)
    mutex.unlock();
}
```

10. Analice el problema de inanición.

Descripción: Para evitar que un flujo masivo de vehículos monopolice indefinidamente el puente (inanición), se implementó un límite máximo de pasos consecutivos (MAX_CONSECUTIVOS = 6), obligando a ceder el turno si el otro lado tiene tráfico en espera.

Archivo: backend/src/main/java/com/puente/backend/servicio/PuenteService.java

Fragmento de código:

```
// RETO: Evitar que una dirección monopolice el puente (Máximo 6 vehículos seguidos)
private static final int MAX_CONSECUTIVOS = 6;
```

```
// MIENTRAS direccion == SUR (o se cumplió el límite de 6 y hay vehículos esperando en el SUR) HACER esperar()
while (estado.getDireccion().equals("SUR") ||
      (estado.getConteoConsecutivoNorte() >= MAX_CONSECUTIVOS && estado.getVehiculosSur() > 0)) {
    mutex.unlock();
    Thread.sleep(200); // esperar()
    mutex.lock();
}
```

```
// MIENTRAS direccion == NORTE (o se cumplió el límite de 6 y hay vehículos esperando en el NORTE) HACER esperar()
while (estado.getDireccion().equals("NORTE") ||
      (estado.getConteoConsecutivoSur() >= MAX_CONSECUTIVOS && estado.getVehiculosNorte() > 0)) {
    mutex.unlock();
    Thread.sleep(200); // esperar()
    mutex.lock();
}
```

Pseudocódigo

MUTEX mutex = 1
direccion = NINGUNA vehiculos_norte = 0
vehiculos_sur = 0
PROCEDIMIENTO vehiculo_norte(id) lock(mutex)

MIENTRAS direccion == SUR HACER esperar()
FIN MIENTRAS
direccion = NORTE
vehiculos_norte = vehiculos_norte + 1 unlock(mutex)
escribir("Vehículo ", id,
" cruza hacia el NORTE")
cruzar_puente(id)
lock(mutex)
vehiculos_norte = vehiculos_norte - 1
SI vehiculos_norte == 0 ENTONCES direccion = NINGUNA
FIN SI
unlock(mutex)
FIN PROCEDIMIENTO

El estudiante deberá completar el procedimiento: PROCEDIMIENTO vehiculo_sur(id)

siguiendo la misma lógica.

Reto: modificar el algoritmo para evitar que una dirección monopolice el puente. Se resolvió utilizando un límite máximo de vehículos consecutivos (MAX_CONSECUTIVOS = 6) junto con contadores de lote (conteoConsecutivoNorte y conteoConsecutivoSur).

Dónde se encuentra en el código:

- **Archivo: backend/src/main/java/com/puente/backend/servicio/PuenteService.java**

Fragmento de código Clave

```
// RETO: Evitar que una dirección monopolice el puente (Máximo 6 vehículos seguidos)
private static final int MAX_CONSECUTIVOS = 6;
```

```
// MIENTRAS direccion == SUR (o se cumplió el límite de 6 y hay vehículos esperando en el SUR) HACER esperar()
while (estado.getDireccion().equals("SUR") ||
      (estado.getConteoConsecutivoNorte() >= MAX_CONSECUTIVOS && estado.getVehiculosSur() > 0)) {
    mutex.unlock();
    Thread.sleep(200); // esperar()
    mutex.lock();
}
```

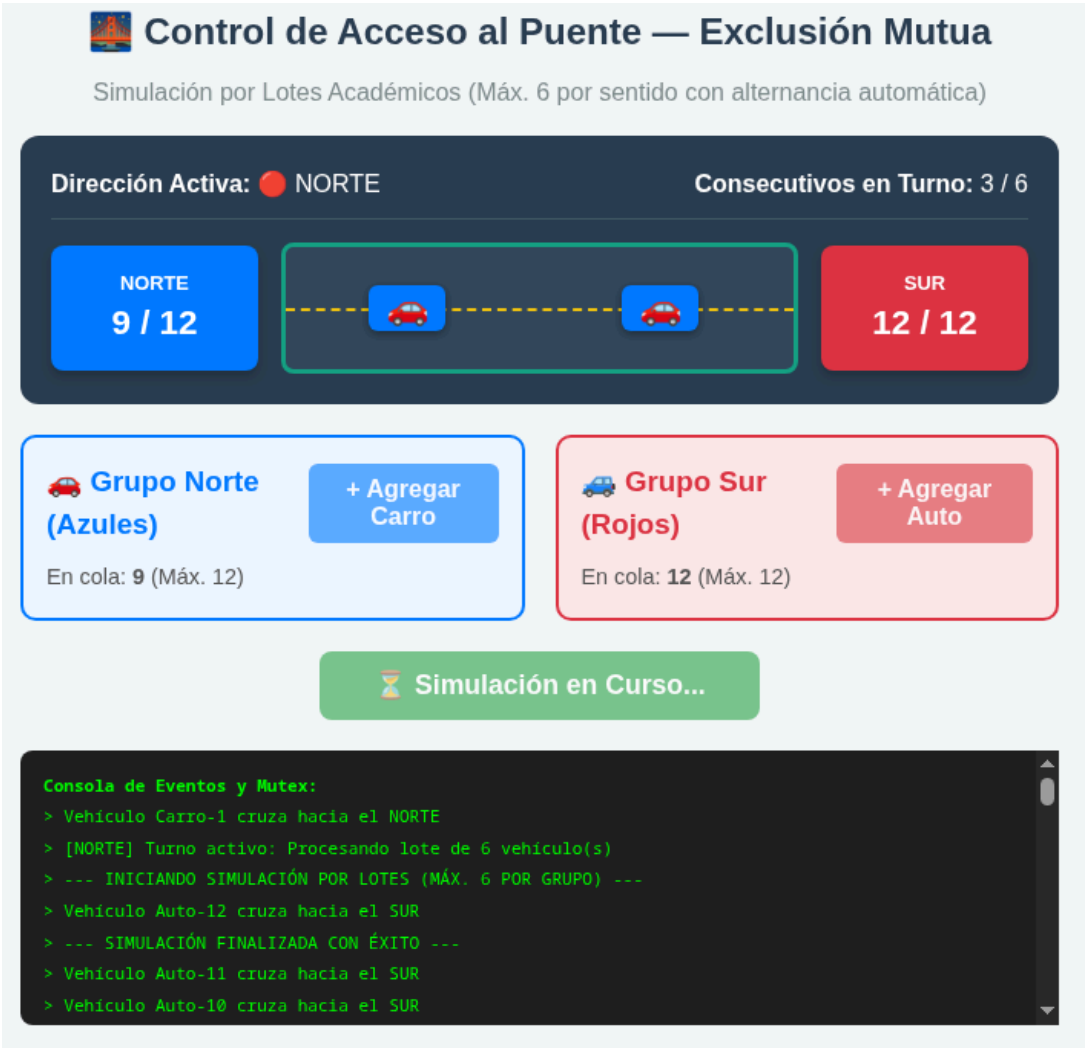
```
// MIENTRAS direccion == NORTE (o se cumplió el límite de 6 y hay vehículos esperando en el NORTE) HACER esperar()
while (estado.getDireccion().equals("NORTE") ||
      (estado.getConteoConsecutivoSur() >= MAX_CONSECUTIVOS && estado.getVehiculosNorte() > 0)) {
    mutex.unlock();
    Thread.sleep(200); // esperar()
    mutex.lock();
}
```

DashBoard

- **Ingreso del numero de autos, máximo 12 por cada grupo con un límite de 6 autos por cruce**



- **Paso del primer grupo de 6_Norte**



- **Paso del primer grupo de 6_Sur**



- Paso del segundo grupo de 6_Norte



- Paso del segundo grupo de 6_Sur



Ejercicio 5. Centro de impresión — Sincronización de procesos

Objetivo: Utilizar semáforos para establecer un orden determinado de ejecución entre procesos.

Situación:

Un centro de impresión tiene tres procesos:

- Diseño
- Revisión
- Impresión

Para cada documento debe cumplirse obligatoriamente: DISEÑO → REVISIÓN → IMPRESIÓN

No se puede revisar un documento que todavía no ha sido diseñado y tampoco se puede imprimir antes de que haya sido revisado.

Procedimiento

1. Identifique los procesos participantes.

   Son tres procesos (hilos): **Diseño**, **Revisión** e **Impresión**. Cada uno trabaja sobre cada documento, y entre los tres deben respetar el orden Diseño -> Revisión -> Impresión.

```
public void start() {
    synchronized (lock) {
        inicioNs = System.nanoTime();
        ultimoProgresoNs = inicioNs;
    }
    pool.submit(() -> proceso(0, turnoDiseno, turnoRevision));
    pool.submit(() -> proceso(1, turnoRevision, turnoImpresion));
    pool.submit(() -> proceso(2, turnoImpresion, turnoDiseno));
    pool.shutdown();
}
```

2. Cree un semáforo para cada etapa.

   Se crean tres semáforos: turnoDiseno, turnoRevision y turnoImpresion. Cada uno funciona como el “permiso” de su etapa: mientras vale 0, esa etapa no puede ejecutarse.

```
private final Semaphore turnoDiseno = new Semaphore(1);
private final Semaphore turnoRevision = new Semaphore(0);
private final Semaphore turnoImpresion = new Semaphore(0);
```

3. Permita inicialmente que el proceso de diseño pueda ejecutarse.

   turnoDiseno se inicializa en 1, mientras que los otros dos empiezan en 0. Así, al arrancar, solo Diseño puede pasar su wait(). Revisión e Impresión quedan bloqueadas esperando.

```
private final Semaphore turnoDiseno = new Semaphore(1);
private final Semaphore turnoRevision = new Semaphore(0)
```

4. Diseño deberá habilitar a revisión.

   Al terminar de diseñar un documento, Diseño hace signal(turno_revision), es decir, suma un permiso a turnoRevision y Revisión puede continuar.

5. Revisión deberá habilitar a impresión.

6. Impresión deberá habilitar nuevamente a diseño.

```
        }
        pool.submit(() -> proceso(0, turnoDiseno, turnoRevision));
        pool.submit(() -> proceso(1, turnoRevision, turnoImpresion));
        pool.submit(() -> proceso(2, turnoImpresion, turnoDiseno));
        pool.shutdown();
    }
```

7. Repita el ciclo para 3 documentos.

   Cada hilo repite su bucle 3 veces, una por documento. Por defecto el frontend arranca con 3 documentos.

```
/** PROCEDIMIENTO diseno() / revision() / impresion(): REPETIR docs VECES */
private void proceso(int e, Semaphore espera, Semaphore habilita) {
    try {
        for (int d = 1; d <= docs; d++) {
            espera.acquire();                          // wait(turno_propio)
            iniciar(e, d);
            Thread.sleep(msEtapa);                     // crear/revisar/imprimir_documento()
            terminar(e, d);
            if (omitir == Etapa.values()[e]) {
                registrar("⚠ " + NOMBRE[e] + " NO ejecuta su signal() (documento " + d + ")");
            } else {
                habilita.release();                    // signal(turno_siguiente)
            }
        }
    } catch (InterruptedException ex) {
        Thread.currentThread().interrupt();
    }
}
```

8. Verifique que el orden siempre sea correcto.

   El programa registra cada etapa en el momento en que arranca y luego compara esa secuencia con la esperada: D1, R1, I1, D2, R2, I2, D3, R3, I3. El orden es correcto porque cada etapa solo avanza cuando recibe el signal() de la anterior.

```
private void iniciar(int e, int doc) {
    synchronized (lock) {
        matriz[doc - 1][e] = 1;
        secuencia.add(PREFIJO[e] + doc);
        ultimoProgresoNs = System.nanoTime();
        log(NOMBRE[e] + " " + VERBO[e] + " documento " + doc);
    }
}
```

9. Analice qué ocurre si se elimina uno de los signal().

El programa entra en interbloqueo (deadlock): algún hilo queda esperando para siempre un permiso que nadie va a dar. Lo que se observa en cada caso:

4 casos:

Normal



signal eliminado para Revisión y Diseño:





Pseudocódigo

SEMÁFORO turno_diseno = 1
SEMÁFORO turno_revision = 0
SEMÁFORO turno_impresion = 0

PROCEDIMIENTO diseno()
REPETIR 3 VECES

wait(turno_diseno)

escribir("Diseño prepara documento")

crear_documento() signal(turno_revision)

FIN REPETIR

FIN PROCEDIMIENTO

PROCEDIMIENTO revision()

REPETIR 3 VECES

wait(turno_revision)

escribir("Revisión verifica documento")

revisar_documento() signal(turno_impresion)

FIN REPETIR

FIN PROCEDIMIENTO

```
PROCEDIMIENTO impresion()

REPETIR 3 VECES

    wait(turno_impresion)

    escribir("Impresión imprime documento")

    imprimir_documento() signal(turno_diseno)

  FIN REPETIR

FIN PROCEDIMIENTO


INICIO

  crear_proceso(diseno)  crear_proceso(revision)

  crear_proceso(impresion)

FIN
```

6. Resultados esperados:

- Comprobación de las distintas formas de como los hilos se comunican.
- Se espera que el estudiante mida y compare el rendimiento entre las soluciones secuenciales y concurrentes
- El estudiante debe demostrar comprensión de los conceptos vistos en cada ejercicio de la practica

Ejercicio	Resultado Esperado	Criterio de Verificación
1. Estacionamiento	Vehículos dentro ≤ 4	Verificar que nunca ingresen más de 4 simultáneamente
2. Cajero	Saldo final = **$650**	Ejecutar varias veces y comprobar que el saldo no varíe
3. Productor- Consumidor	Todos los productos producidos son consumidos	Verificar que el almacén nunca supere 5 ni quede con inconsistencias
4. Puente	Solo una dirección activa a la vez	Comprobar que nunca crucen vehículos en sentidosopuestos simultáneamente
5. Centro de Impresión	Diseño → Revisión → Impresión	Verificar que ningún proceso avance a la siguiente etapa antes de tiempo

7. Preguntas de Control:

- **¿Qué es una condición de carrera?**

  Una condición de carrera ocurre cuando dos o más hilos acceden al mismo recurso compartido de manera simultánea y el resultados dependen del orden en el que se llegan a ejecutar. lo que puede llegar a causar que los resultados finales en los sistemas no sean precisos y equívocos.
- **Cuál es la diferencia fundamental entre un semáforo binario y un mutex? ¿Pueden usarse indistintamente? Justifique su respuesta con al menos dos diferencias prácticas**

  El mutex sirve para proteger un recurso compartido, mientras que un semáforo binario permite el control de acceso o sincronización entre hilos mediante un valor de 0 y1.
  propiedad: mutex se libera únicamente por el hilo que lo bloqueo, un semáforo binario puede ser liberado por otro hilo
  finalidad: mutex protege secciones críticas, mientras que el semáforo binario puede notificar a otro hila la finalización de una tarea ha finalizado
- **Explique qué es un interbloqueo (deadlock) y enuncie las cuatro condiciones necesarias de Coffman. ¿Cuál de estas condiciones se elimina al usar la estrategia de ordenamiento de recursos?**
  Un interbloqueo ocurre cuando dos o más hilos quedan esperando indefinidamente recursos que se están bloqueando por otro, lo que impide la continuación de la ejecución

  Exclusión mutua: un recurso solo puede ser usado por un hilo a la vez
  Retención y espera: un hilo mantiene un recurso mientras espera obtener otro
  No expropiación: Un recurso no puede retirarse por fuerza al hilo que lo posee
  Espera circular: existe una cadena de hilos en la que cada uno espera un recurso retenido por el siguiente

8. Conclusiones

- La presente práctica permite aplicar e implementar mutex, en el caso del ejercicio 2, se usó para la protección del saldo becario compartido, además de evitar que se presenten las ya conocidas condiciones de carrera, bajo el escenario de retiro y depósito. Comprobando que la exclusión mutua garantiza que las operaciones se realicen de forma segura.

- Se comprobó el como los semáforos y el mutex permiten resolver el problema planteado en el tercer ejercicio, mediante los semáforos ‘espacios’ y ‘elementos’ se controló el flujo, ya sea bloqueando el productor o consumidor dependiendo de la situación, por su parte es fundamental aplicar mutex ya que este evita condiciones de carrera, lo cual se evidencio a la hora de insertar o retirar piezas, por ende la coordinación entre recursos y exclusión mutua es clave a la hora de diseñar sistemas concurrentes.
- Con los ejercicios 1 y 5 comprobamos que los semáforos son una herramienta de sincronización con dos usos distintos. En el ejercicio 1, el semáforo contador inicializado en 4 controló el acceso a un recurso limitado. Aunque llegaron 10 vehículos a la vez, nunca hubo más de 4 dentro del estacionamiento: los demás quedaron bloqueados en wait() sin consumir CPU, y cada signal() al salir despertó a uno de ellos. En el ejercicio 5, los semáforos no controlaron cuántos procesos entran, sino en qué orden actúan. Con turnoDiseno en 1 y los otros dos en 0, cada etapa solo avanza cuando la anterior le da el permiso, y el ciclo se cierra cuando Impresión habilita de nuevo a Diseño. Así el orden Diseño -> Revisión -> Impresión se cumplió en todas las ejecuciones.
- La práctica me permitió entender acerca de gestión adecuada a un acceso concurrente a un recurso compartido del puente de carril único, dando una solución a partir de alternancia del carril equitativa y segura entre grupos opuestos, sincronizando de forma óptima el estado del sistema previniendo así condiciones de carrera. Poner en práctica estas actividades fortalecen el diseño de sistemas de este tipo de forma adecuada y que

enriquezca nuestro conocimiento acerca del mismo.

9. Declaración de uso de IA

Para la presente práctica se usó la herramienta NotebookLM como una herramienta de consulta para dar respuesta a dudas técnicas que se plantearon durante la práctica y usamos ChatGPT y Claude para ayudarnos como una guía para el desarrollo de los ejercicios, con las herramientas usadas se puede afirmar que usamos como apoyo, con responsabilidad y honestidad.

PROMPTS UTILIZADOS

Prompt usado: Actúa como tutor académico especializado en Sistemas Distribuidos.Utiliza únicamente la información disponible en NotebookLM para responder preguntas técnicas sobre la práctica de exclusión mutua, condiciones de carrera, mutex, semáforos binarios e interbloqueos.

Ejercicio 1

Según la guía adjunta, resuelve el Ejercicio 1 (estacionamiento con semáforo contador). Hazlo con Spring Boot (Java 17) para el backend y React con Vite para el frontend. La interfaz debe mostrar los espacios ocupados, los vehículos en espera, el máximo de vehículos dentro y si cumple el criterio de verificación (nunca más de 4).

- **Ejercicio 2**

  Desarrolla el ejercicio de Sistemas Distribuidos "Cajero automático compartido — Exclusión mutua" usando Java con Spring Boot para el backend y React con Vite para el frontend.

  Crea una cuenta bancaria con un saldo inicial de $1000 y tres operaciones concurrentes: un retiro de $300, otro de $250 y un depósito de $200.
  Utiliza `ReentrantLock` para proteger el saldo compartido y compara la ejecución con y sin el uso de mutex.

  Muestra en una interfaz sencilla el saldo inicial, las operaciones realizadas y el saldo final.
  El resultado esperado al usar mutex es de $650.El código debe ser simple, bien organizado y fácil de entender.

  Trabaja con un proyecto existente, sin utilizar Docker ni base de datos, y explica cómo ejecutarlo localmente.

- **Ejercicio 3:**

  Resuelve el Ejercicio 3 (Fábrica de ensamblaje con semáforos: Productor-Consumidor con cola limitada de 5 piezas),

  Requisitos de Backend (Spring Boot - Java 17):

  Crea la lógica en el paquete com/fabrica/simulacion/core/ utilizando java.util.concurrent.Semaphore y un objeto Mutex (o ReentrantLock) para la exclusión mutua.

  Implementa una cola limitada con capacidad máxima de 5.

  Define Semaphore espacios = new Semaphore(5) y Semaphore elementos = new Semaphore(0).

  Desarrolla los hilos Productor y Consumidor asegurando el patrón: wait(espacios) -> acquire(mutex) -> insertar/retirar -> release(mutex) -> signal(elementos/espacios).

Expón endpoints REST en SimulacionController y EventosController para iniciar/detener la simulación y consultar el estado actual de la cola y los hilos.

Requisitos de Frontend (Flask + HTML/CSS/JS en frontend-flask/):

Crea una interfaz visual (usando templates/index.html y static/js/app.js) que consuma los endpoints del backend.

Muestra en tiempo real: el estado de la cola (0 a 5 piezas), el log de eventos (quién produjo/consumió) y el estado de los semáforos.

- **Ejercicio 4**

""Hola, necesito tu ayuda para elaborar la documentación técnica y la configuración de arquitectura para un proyecto web full-stack de simulación concurrente.

Por favor, genera una guía detallada y profesional que incluya los siguientes puntos:

1. **Tecnologías Utilizadas (Tech Stack):**
   - **Backend:** Descripción técnica de Java 17+, Spring Boot 3.x, Maven y el uso del paquete `java.util.concurrent.locks` (`ReentrantLock`) para controlar la exclusión mutua.
   - **Frontend:** Descripción técnica de React 18+, Vite, manejo de estados, efectos y animaciones en CSS nativo con `@keyframes`.
   - **Despliegue:** Breve resumen del uso de Docker y Docker Compose.

2. **Organización de Carpetas y Archivos:**
   - Estructura clara de directorios en formato árbol mostrando la separación entre `/backend`, `/frontend` y la raíz del proyecto.
   - Ubicación y propósito de las clases clave del backend (`PuenteController`, `PuenteService`, `Puente`, `CorsConfig`) y componentes del frontend (`App.jsx`).

3. **Comunicación entre Backend y Frontend:**
   - Explicación de la arquitectura RESTful y el protocolo HTTP (`GET` para polling de estado en tiempo real, `POST` para llamadas bloqueantes/síncronas de hilos).
   - Cómo se resuelven las políticas CORS en Spring Boot.

4. **Guía de Ejecución Local:**
   - Prerrequisitos de instalación (JDK, Node.js).
   - Pasos y comandos de terminal para clonar, instalar dependencias y levantar el servidor backend (`port 8080`) y frontend (`port 5173`).

5. **Dockerización (Despliegue Multi-Contenedor):**
   - El código completo del `Dockerfile` para el Backend (Spring Boot con compilación multi-stage).
   - El código completo del `Dockerfile` para el Frontend""

Ejercicio 5

Según la guía adjunta, resuelve el Ejercicio 5 (centro de impresión con semáforos: Diseño → Revisión → Impresión, para 3 documentos). Hazlo con Spring Boot (Java 17) para el backend y React con Vite para el frontend. La interfaz debe mostrar el avance de cada documento por etapa, la secuencia de ejecución y si el orden es correcto. Agrega una opción para eliminar un signal() y ver el interbloqueo, para el punto 9 del procedimiento.

10. Evaluación

Criterio	Excelente	Bueno	Bajo
Funcionamiento (back-end, front-end) se evaluara criterios de Verificacion	Funciona completo (4)	Parcial (2)	No funciona (0)
Documento de practica (Enlace a git, metodología usada por el alumno, resultados, preguntas de control, bibliografía, conclusiones, video de explicación)	Funciona completo (3)	Parcial (2)	No funciona (0)
Defensa de practica	Todos los puntos (3)	Parcial (1)	No valido (0)

11. Bibliografía

Semana 3. Guaman, Jose. 2026

12. Elaboración y Aprobación

Elaborado por	José Guamán **Docente**	Validar únicamente en FirmaEC. Firmado electrónicamente por: **JOSE OSWALDO GUAMAN QUINCHE**
Ejecutado Por	Rene Rivera **Técnico Docente**	Validar únicamente en FirmaEC. Firmado electrónicamente por: **RAMIRO RENE RIVERA GUAMAN**
Aprobado por	Edison L Coronel Romero **Director de Carrera**	Firmado electrónicamente por: **EDISON LEONIDAS CORONEL ROMER** Validar únicamente con FirmaEC